Mini Project 9. Graph Algorithms Explorer

FOUNDATION-FOCUSED

An interactive toolkit that builds graphs and computes their structural properties and classic algorithms.

Overview

What is it?

- Create and edit graphs (directed/undirected, weighted).
- Traversals, shortest paths, spanning trees, connectivity, and cycles.
- Structural metrics: degree distribution, centrality, components, colouring.
- Visualise results on sample networks and mazes.

Foundational courses covered

- Discrete Structures & Computational Thinking (graph theory).
- Data Structures & Algorithms (graph representations and algorithms).
- Python Programming & OOP (graph model).

Technologies

Python (with a plotting/graph library), or C++ 9 / 18 · Foundation-focused

Architecture & Design

Algorithms

- BFS, DFS, connected components
- Dijkstra and Bellman–Ford shortest paths
- Kruskal / Prim minimum spanning tree
- Topological sort and cycle detection
- Graph colouring and bipartite test

Data structures

- Adjacency list and adjacency matrix
- Priority queue / min-heap (Dijkstra)
- Union-Find (disjoint sets)
- Queues (BFS) and stacks (DFS)
- Hash maps for visited / distances

Architecture & design concepts

- Layered: graph model, algorithms, visualization
- Strategy pattern for interchangeable algorithms
- Representation separated from traversal
- Pluggable graph representations

Exclusions

- Graph libraries (NetworkX, igraph)
- Graph databases
- Web frameworks and cloud services
- Machine Learning or AI libraries

Worklets & Learning-Outcome Checks

Worklet 1: Graph model, traversal & connectivity (DSA + Discrete)

- Implement adjacency-list and adjacency-matrix representations (directed/undirected/weighted).
- Build editing operations and load graphs from an edge-list file.

- Implement BFS and DFS with visit ordering.
- Find connected (and strongly connected) components; detect cycles.
- Compute a topological order for a DAG; test bipartiteness by 2-colouring.

Worklet 2: Weighted-graph algorithms

- Implement Dijkstra's shortest path with a priority queue.
- Implement Bellman–Ford and detect negative cycles.
- Implement a minimum spanning tree (Kruskal or Prim) with union-find.
- Verify results against brute force on small graphs.
- Compare the space/time trade-offs of the two representations.

Worklet 3: Structural analysis, visualisation & evaluation

- Compute degree distributions and a centrality measure.
- Greedily colour the graph; generate and solve mazes as graphs.
- Render graphs and highlight algorithm outputs (paths, trees).
- Animate a traversal step by step.
- Run on a real-ish dataset and benchmark as size grows.

Learning-outcome checks (verify understanding, not just “did it run”)

- When is an adjacency matrix better than a list, and why?
- Why does Dijkstra fail with negative edges, and what does Bellman–Ford do instead?
- Prove a graph is bipartite iff it has no odd cycle, and show your test detects it.
- What does union-find give Kruskal, and what is its near-constant cost?
- Trace BFS from a node and explain why it yields shortest paths in an unweighted graph.